

模糊综合评价模型建立方法

——从数据处理到结果可视化的全流程学习笔记

冯浩然

配套项目: `fuzzy_evaluation.ipynb` (Jupyter Notebook)

数据文件: `company_data.xlsx` (7 公司 × 19 指标)

2026 年 7 月 9 日

目录

1	什么时候用模糊综合评价?	3
1.1	适用场景判断（满足两条以上即适用）	3
1.2	不适用场景	3
2	方法总览：一张图看懂全过程	4
3	第一步：构建评价指标体系	4
3.1	三层结构模板	4
3.2	设计原则	5
3.3	指标类型分类（必须确定）	5
3.4	本项目指标体系	6
4	第二步：准备与标准化数据	6
4.1	数据文件组织（Excel 三工作表结构）	6
4.2	评分标准化方法	6
4.3	代码读取 Excel	6
5	第三步：设定评价等级与隶属函数	7
5.1	五级评价标尺	7
5.2	梯形隶属函数的几何含义	8
5.3	三种梯形函数的 Python 实现	8
5.4	参数设置（最需人工判断的一步）	9
5.5	计算隶属度矩阵	9
5.6	隶属度计算实例	9
6	第四步：用 AHP 确定权重	10
6.1	AHP 是什么?	10
6.2	1~9 标度法	10
6.3	方根法求权重（三步走）	10
6.4	Python 实现	11
6.5	本项目的四个判断矩阵	11
6.6	多层权重合成	12
7	第五步：执行模糊综合评价	12
7.1	两级合成的逻辑	12
7.2	合成算子： $M(\cdot, \oplus)$ 加权平均型	12
7.3	完整计算实例：公司 a	13
7.4	七家公司最终排名	14
8	第六步：结果可视化	14
8.1	各图设计要点	14

9 代码架构与复用指南	15
9.1 项目文件结构	15
9.2 Notebook 模块划分	16
9.3 复用到自己的项目需要改什么?	16
10 常见问题与避坑指南	16
10.1 Q1: 隶属函数参数怎么选?	16
10.2 Q2: $CR > 0.1$ 怎么办?	17
10.3 Q3: 为什么隶属度矩阵里有全 1 或全 0 的行?	17
10.4 Q4: 能改成四等级或七等级吗?	17
10.5 Q5: 四种模糊合成算子的区别?	17
10.6 Q6: 如何做灵敏度分析?	17
附录: 项目文件清单	18

1 什么时候用模糊综合评价？

1.1 适用场景判断（满足两条以上即适用）

1. 指标方向不一致：有的越大越好（利润），有的越小越好（成本），有的越接近中间越好（资产负债率）。
2. 评价等级有模糊边界：不能严格说“90 分是优秀、89 分是良好”——边界是渐变的。
3. 权重需要系统化确定：指标重要性不同，不能拍脑袋分配权重。
4. 需要给多个对象排序：最终要排出优劣顺序。

1.2 不适用场景

- 数据有明确、无争议的二元判别标准（如“是否通过认证”）。
- 只需要给出一个对象的单一定性结论（如“好/不好”）。
- 指标之间高度线性相关，用简单加权平均即可。

核心概念

模糊综合评价（Fuzzy Comprehensive Evaluation, FCE）的**核心优势**是：用隶属函数将精确指标值转化为对多个评价等级的隶属程度（0~1 之间），然后用层次分析法（AHP）系统确定权重，最后加权合成为综合得分。

2 方法总览：一张图看懂全过程

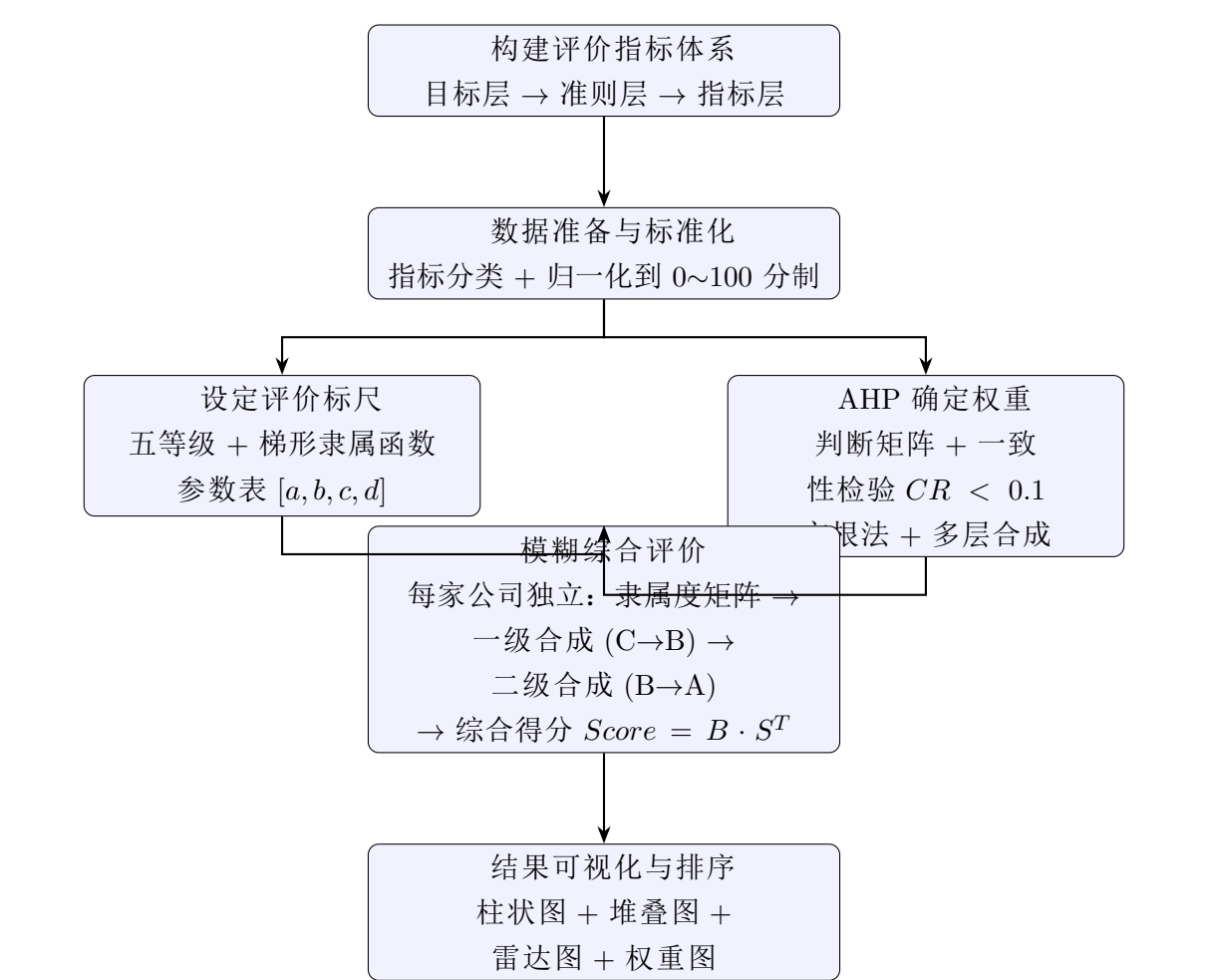


图 1：FCE 方法全流程

核心概念

核心思想：FCE 不是把对象之间相互比较，而是让每个对象独立面对同一把”评价尺子”（五等级标尺），用隶属函数度量它在每个等级上的程度，再用权重合成总分。

3 第一步：构建评价指标体系

3.1 三层结构模板

表 1：评价指标体系的标准三层结构

层级	别称	作用	本项目示例
目标层（A）	最终目标	你要评价什么	公司综合实力
准则层（B）	一级指标	从哪几个大方面评价	B ₁ 产品、B ₂ 成本、B ₃ 销售
指标层（C）	二级指标	每个方面的具体指标	C ₁ ~C ₁₉ 共 19 个

3.2 设计原则

- 1. 系统性：B 层覆盖 A 层的主要维度，C 层覆盖 B 层的每个方面
- 2. 可操作性：每个 C 层指标能获取数据（真实或模拟均可）
- 3. 层次性：同层指标重要性有差异但不能差一个数量级
- 4. 独立性：同层指标间信息重叠尽量少

实操技巧

- B 层数量：通常 3~5 个。太多则层次优势丧失，太少则 C 层过长
- C 层数量：每个 B 下 3~8 个，总 C 层 10~25 个为宜
- 确定指标后立即列出表格，标明每个 C 层指标的类型

3.3 指标类型分类（必须确定）

每个二级指标归入以下三类之一，这决定后续用什么隶属函数：

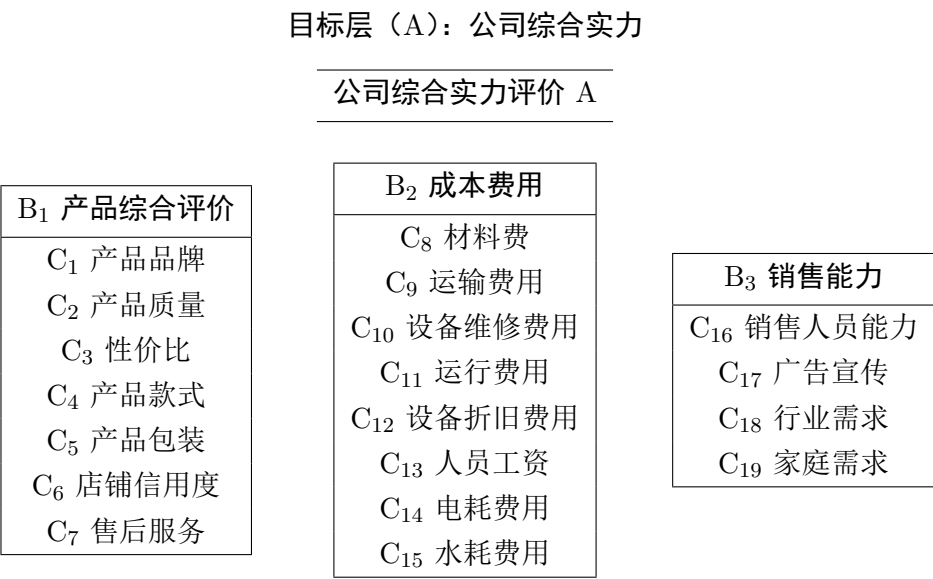
表 2: 三种指标类型

类型	通俗理解	数学特征	本项目数量
偏大型（效益型）	越大越好	高分 → 高级别	9 个
偏小型（成本型）	越小越好	低分 → 高级别	8 个
中间型（适度型）	中间最好	最优区间中心 → 高级别	2 个

实操技巧

不确定类型时问自己：“这个指标的理想值在什么位置？”答案在高位 → 偏大型；在低位 → 偏小型；在中间 → 中间型。

3.4 本项目指标体系



```
4         sheet_name='数据(仅数值)', index_col=0)
5 df_meta = pd.read_excel('company_data.xlsx',
6         sheet_name='指标体系')
7
8 # 构建 "指标名 -> 类型" 映射
9 indicator_types = dict(zip(df_meta['二级指标'], df_meta['指标类型']))
10
11 companies = df_raw.columns.tolist()      # ['a', 'b', ..., 'g']
12 indicators = df_raw.index.tolist()       # ['C1_...', ..., 'C19_...']
```

核心概念

indicator_types 字典是隶属函数计算的调度中心，代码通过它自动判断每个指标该用偏大型、偏小型还是中间型函数。

5 第三步：设定评价等级与隶属函数

这是 FCE 区别于普通加权评分的最核心步骤。

5.1 五级评价标尺

表 4: 五级评价等级与分值

等级 v_i	v_1 优秀	v_2 良好	v_3 中等	v_4 较差	v_5 很差
分值 s_i	100	80	60	40	20

避坑提醒

为什么不用三级或七级？

- 三级太粗糙，区分度不够
- 七级以上相邻等级语义重叠严重
- 五级与百分制每 20 分一档自然对应，是领域内最成熟的设定

5.2 梯形隶属函数的几何含义

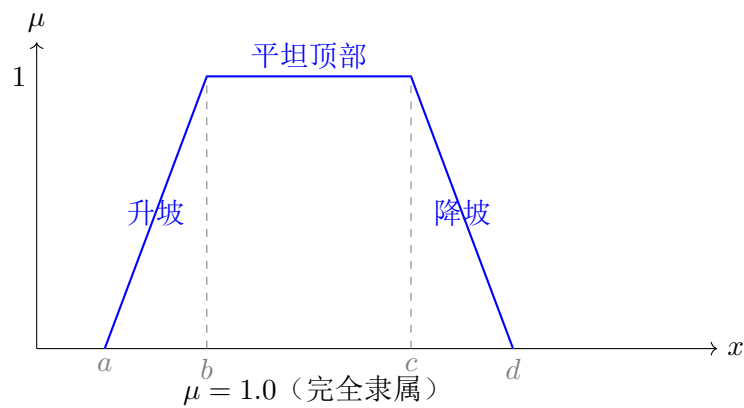


图 3: 梯形隶属函数的四参数 $[a, b, c, d]$

为什么选梯形？

- 三角形（两端尖）→ 完全隶属只有一个点，对数据太敏感
- 梯形（顶部平坦）→ 有一个稳定区间，更符合直觉
- 高斯型 → 参数多、难以解释

5.3 三种梯形函数的 Python 实现

```

1 def trapezoid_up(x, a, b, c, d):
2     """偏大型：效益型指标，x 越大隶属度越高"""
3     if x <= a:         return 0.0
4     elif x <= b:       return (x - a) / (b - a)
5     elif x <= c:       return 1.0
6     elif x <= d:       return (d - x) / (d - c)
7     else:              return 0.0
8
9 def trapezoid_down(x, a, b, c, d):
10    """偏小型：成本型指标，x 越小隶属度越高"""
11    if x <= a:          return 1.0
12    elif x <= b:        return (b - x) / (b - a)
13    elif x <= c:        return 0.0
14    elif x <= d:        return (x - c) / (d - c)
15    else:               return 0.0
16
17 def trapezoid_mid(x, a, b, c, d):
18    """中间型：在区间 [b, c] 内最优"""
19    if x <= a:          return 0.0
20    elif x <= b:        return (x - a) / (b - a)
21    elif x <= c:        return 1.0
22    elif x <= d:        return (d - x) / (d - c)
23    else:               return 0.0

```

5.4 参数设置（最需人工判断的一步）

表 5：五等级 × 三类型的 15 组隶属函数参数 $[a, b, c, d]$

等级	分值	偏大型	偏小型	中间型
v_1 优秀	100	(80, 90, 100, 100)	(0, 0, 15, 25)	(65, 75, 85, 95)
v_2 良好	80	(70, 80, 80, 90)	(15, 25, 25, 35)	(55, 65, 65, 75)
v_3 中等	60	(60, 70, 70, 80)	(25, 35, 35, 45)	(45, 55, 55, 65)
v_4 较差	40	(50, 60, 60, 70)	(35, 45, 45, 55)	(35, 45, 45, 55)
v_5 很差	20	(0, 0, 50, 60)	(45, 55, 100, 100)	(0, 0, 35, 45)

核心概念

参数设置的黄金法则：确保相邻等级之间有重叠过渡区。

以偏大型”优秀” $[80, 90, 100, 100]$ 和”良好” $[70, 80, 80, 90]$ 为例：两者在 80~90 区间形成对称过渡——在 85 分处，对”优秀”的隶属度为 $(85 - 80)/10 = 0.5$ ，对”良好”的隶属度为 $(90 - 85)/10 = 0.5$ ，恰好各半，体现了”边界模糊”特性。

5.5 计算隶属度矩阵

```
1  GRADE_KEYS = ['v1_优秀', 'v2_良好', 'v3_中等',
2               'v4_较差', 'v5_很差']
3
4  def calc_membership(value, itype):
5      """对单个指标值，返回其对5个等级的隶属度向量"""
6      if itype == '偏大型':
7          func, params = trapezoid_up, GRADE_PARAMS_UP
8      elif itype == '偏小型':
9          func, params = trapezoid_down, GRADE_PARAMS_DOWN
10     else:
11         func, params = trapezoid_mid, GRADE_PARAMS_MID
12     return np.array([func(value, *params[gk]) for gk in GRADE_KEYS])
13
14 # 对每家公司、每个指标逐一计算
15 R_matrices = {}
16 for comp in companies:
17     R_matrices[comp] = {}
18     for ind in indicators:
19         value = df_raw.loc[ind, comp]
20         R_matrices[comp][ind] = calc_membership(value, indicator_types[ind])
```

5.6 隶属度计算实例

示例 1：C₂ 产品质量 ($x = 94.7$ ，偏大型)

- 优秀 $[80, 90, 100, 100]$: $94.7 \in [90, 100] \Rightarrow \mu = 1.000$
- 良好 $[70, 80, 80, 90]$: $94.7 > 90 \Rightarrow \mu = 0.000$

- 中等/较差/很差：同理 $\mu = 0.000$

结果：[1.000, 0.000, 0.000, 0.000, 0.000]——完全隶属”优秀”。

示例 2: C_3 性价比 ($x = 87.0$, 偏大型) ——展示边界过渡

- 优秀 [80, 90, 100, 100]: $87.0 \in [80, 90)$, $\mu = (87 - 80)/(90 - 80) = 0.700$
- 良好 [70, 80, 80, 90]: $87.0 \in (80, 90]$, $\mu = (90 - 87)/(90 - 80) = 0.300$
- 中等/较差/很差: $\mu = 0.000$

结果：[0.700, 0.300, 0.000, 0.000, 0.000]——70% 优秀 + 30% 良好。

核心概念

这就是 FCE 区别于”一刀切”评分的核心：87 分的性价比不是被强行归入”优秀”或”良好”，而是精确保留了在两者之间的模糊过渡状态。这种信息在后续加权后会体现在综合得分的小数部分。

6 第四步：用 AHP 确定权重

6.1 AHP 是什么？

层次分析法（Analytic Hierarchy Process）由 Saaty 于 1977 年提出，核心思想：将 n 个指标的权重问题转化为 $n(n-1)/2$ 次两两比较，用数学方法从判断矩阵中提取一致性的权重向量。

6.2 1~9 标度法

表 6: 1~9 标度含义

标度	含义
1	两个因素同等重要
3	前者比后者稍微重要
5	前者比后者明显重要
7	前者比后者强烈重要
9	前者比后者极端重要
2,4,6,8	相邻判断的中间值
倒数	$a_{ji} = 1/a_{ij}$ （反向比较）

6.3 方根法求权重（三步走）

Step 1: 构造判断矩阵

用 1~9 标度法两两比较，填满 $n \times n$ 的正互反矩阵 A 。对角线全为 1，对立位置互为倒数。

Step 2: 方根法计算权重

每行求几何平均 → 归一化:

$$w_i = \frac{(\prod_{j=1}^n a_{ij})^{1/n}}{\sum_{i=1}^n (\prod_{j=1}^n a_{ij})^{1/n}}$$

选方根法而非特征向量法的原因: 计算简便且精度几乎相同。

Step 3: 一致性检验

$$\lambda_{\max} = \frac{1}{n} \sum_{i=1}^n \frac{(AW)_i}{w_i}, \quad CI = \frac{\lambda_{\max} - n}{n - 1}, \quad CR = \frac{CI}{RI}$$

要求 $CR < 0.1$, 否则需调整判断矩阵。

6.4 Python 实现

```

1 def ahp_weights(judgment_matrix, method='geometric_mean'):
2     n = judgment_matrix.shape[0]
3     geo_mean = np.power(np.prod(judgment_matrix, axis=1), 1.0 / n)
4     weights = geo_mean / np.sum(geo_mean)
5     Aw = judgment_matrix @ weights
6     lambda_max = np.mean(Aw / weights)
7     CI = (lambda_max - n) / (n - 1) if n > 1 else 0.0
8     RI_table = {1:0, 2:0, 3:0.58, 4:0.90, 5:1.12, 6:1.24,
9                 7:1.32, 8:1.41, 9:1.45, 10:1.49}
10    RI = RI_table.get(n, 1.5)
11    CR = CI / RI if RI > 0 else 0.0
12    return weights, lambda_max, CR

```

6.5 本项目的四个判断矩阵

本项目共需构造 1（一级）+ 3（二级）= 4 个判断矩阵。

一级矩阵 $A \rightarrow B$ (3×3):

A	B ₁ 产品	B ₂ 成本	B ₃ 销售	权重
B ₁ 产品	1	2	3	0.5396
B ₂ 成本	1/2	1	2	0.2970
B ₃ 销售	1/3	1/2	1	0.1634

$$\lambda_{\max} = 3.0092, \quad CI = 0.0046, \quad CR = 0.0079 < 0.1 \quad \checkmark$$

二级矩阵 (3 个): $B_1 \rightarrow C$ (7×7): C_2 质量 = C_3 性价比 (0.2468) > C_6 信用 = C_7 售后 (0.1396) > $C_1=C_4=C_5$ (0.0758)

$B_2 \rightarrow C$ (8×8): C_8 材料费最高 (0.2499) > $C_9 \sim C_{12}$ 运营费 (0.1433) > C_{13} 工资 (0.0798) > $C_{14}=C_{15}$ (0.0485)

$B_3 \rightarrow C$ (4×4): C_{16} 销售能力最高 (0.4231) > $C_{18}=C_{19}$ 需求 (0.2274) > C_{17} 广告 (0.1222)
全部 $CR < 0.1$, 通过一致性检验。

避坑提醒

CR 偏高怎么办？ 找出判断矩阵中的”三角矛盾”（如 $A > B$, $B > C$, 但 $C > A$ ），修改最不合理那条判断，重新计算 CR 直到 < 0.1 。3×3 矩阵通常自然满足，7×7 以上容易偏高，需仔细检查。

6.6 多层权重合成

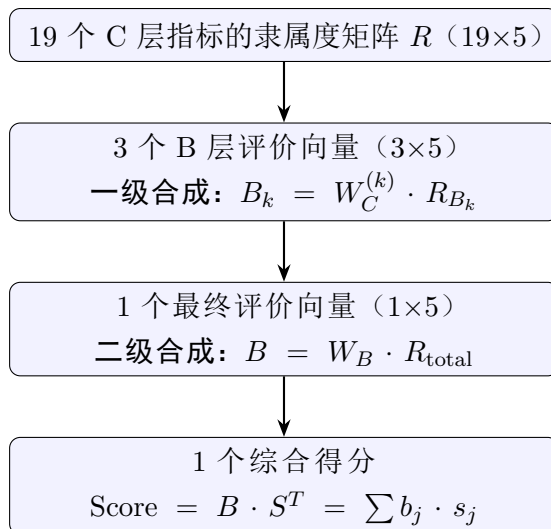
$$w_i^{\text{综合}} = \frac{w_B^{(i)} \cdot w_C^{(i)}}{\sum_k w_B^{(k)} \cdot w_C^{(k)}}$$

```

1 combined = np.concatenate([
2     w_B[0] * w_C1,    # B1 下 7 个指标
3     w_B[1] * w_C2,    # B2 下 8 个指标
4     w_B[2] * w_C3,    # B3 下 4 个指标
5 ])
6 combined = combined / np.sum(combined)

```

综合权重前三名： C_3 性价比 (13.32%) = C_2 产品质量 (13.32%) > C_6 店铺信用度 (7.53%)。集中在产品维度，与该维度一级权重 53.96% 一致。

7 第五步：执行模糊综合评价**7.1 两级合成的逻辑****7.2 合成算子：M(·,⊕) 加权平均型**

$$b_j = \sum_{i=1}^n w_i \cdot r_{ij} \quad (j = 1, \dots, 5)$$

选择理由：权重与隶属度逐项相乘后求和，**不丢弃任何信息**。每个指标的每个等级隶属度都参与了最终结果的计算。

避坑提醒

其他算子的局限性：

- $M(\wedge, \vee)$ （取小取大）——只用最突出因素的信息，损失严重
- $M(\cdot, \vee)$ （乘积取大）——丢失非最大项的贡献
- $M(\cdot, \oplus)$ （加权平均）——保留全部信息，是默认首选

7.3 完整计算实例：公司 a

一级合成—— B_1 产品维度：

$$\begin{aligned}
 B_1^a &= [0.0758, 0.2468, 0.2468, 0.0758, 0.0758, 0.1396, 0.1396] \\
 &\quad \cdot \begin{bmatrix} 1.000 & 0.000 & 0 & 0 & 0 \\ 1.000 & 0.000 & 0 & 0 & 0 \\ 0.700 & 0.300 & 0 & 0 & 0 \\ 0.910 & 0.090 & 0 & 0 & 0 \\ 0.030 & 0.970 & 0 & 0 & 0 \\ 1.000 & 0.000 & 0 & 0 & 0 \\ 0.690 & 0.310 & 0 & 0 & 0 \end{bmatrix} \\
 &= [0.802, 0.198, 0.000, 0.000, 0.000]
 \end{aligned}$$

→ 产品维度 80.2% 隶属”优秀”，19.8% 隶属”良好”。

同理得 B_2 成本和 B_3 销售：

$B_2^a = [0.046, 0.190, 0.253, 0.255, 0.255] \rightarrow$ 最大等级：较差

$B_3^a = [0.000, 0.480, 0.520, 0.000, 0.000] \rightarrow$ 最大等级：中等

二级合成——公司整体评价向量：

$$\begin{aligned}
 B_a &= 0.5396 \times [0.802, 0.198, 0, 0, 0] \\
 &\quad + 0.2970 \times [0.046, 0.190, 0.253, 0.255, 0.255] \\
 &\quad + 0.1634 \times [0, 0.480, 0.520, 0, 0] \\
 &= [0.447, 0.242, 0.160, 0.076, 0.076]
 \end{aligned}$$

→ 公司整体 44.7% 隶属”优秀”（最高）。

综合得分：

$$\begin{aligned}
 \text{Score}_a &= 0.447 \times 100 + 0.242 \times 80 + 0.160 \times 60 + 0.076 \times 40 + 0.076 \times 20 \\
 &= 44.7 + 19.4 + 9.6 + 3.0 + 1.5 = \mathbf{78.15}
 \end{aligned}$$

7.4 七家公司最终排名

表 7：七家公司最终模糊评价结果及排序

排名	公司	评价向量（优，良，中，差，很差）	等级	得分
1	d	(0.505, 0.270, 0.077, 0.074, 0.074)	优秀	81.14
2	a	(0.447, 0.242, 0.160, 0.076, 0.076)	优秀	78.15
3	c	(0.183, 0.428, 0.227, 0.081, 0.081)	良好	71.02
4	e	(0.276, 0.300, 0.183, 0.154, 0.087)	良好	70.50
5	b	(0.186, 0.326, 0.297, 0.115, 0.076)	良好	68.66
6	f	(0.080, 0.390, 0.315, 0.120, 0.095)	良好	64.79
7	g	(0.127, 0.229, 0.415, 0.128, 0.101)	中等	63.04

核心概念

d 的胜出原因：三个维度均衡。产品维度优秀（0.787）、成本维度七家中最佳（0.243）、销售维度良好（0.506）。结合原始数据，d 的成本指标值（ $C_8 \sim C_{15}$ ）全面低于对手，尤其是材料费（8.6 vs 行业均值 ≈ 28 ），形成了显著的成本护城河。

8 第六步：结果可视化

本项目的 5 张图与它们各自回答的问题：

表 8：五张可视化图表的功能说明

图表	文件名	回答的问题
图 1 综合得分柱状图	score_ranking.png	谁是第一、谁最后？差距多大？
图 2 等级隶属度堆叠图	grade_distribution.png	每家公司”优秀/良好/...”占比？
图 3 一级指标雷达图	radar_chart.png	每家在三个维度上谁强谁弱？
图 4 一级指标分组图	dimension_comparison.png	所有公司在同一维度的直接对比
图 5 权重分布图	weight_distribution.png	19 个指标中谁的权重最大/最小？

8.1 各图设计要点

图 1（得分柱状图）：最高分红标；四条等级分界线（85/70/50/35）用虚线标注。

图 2（隶属度堆叠图）：颜色从绿（优秀）→ 红（很差），形成视觉梯度；每段 >5% 时才标注数字。

图 3（雷达图）：三轴对应三个 B 层维度；7 条不同颜色的线 = 7 家公司的”三维画像”。用 subplot_kw=dict(polar=True) 创建极坐标。

图 4（分组柱状图）：每组 3 根柱子，三种颜色对应三个维度，白色数字标注在柱内（值 >15 时才标注）。

图 5（权重图）：水平条形图，按权重升序排列；三色编码（红 >6%，橙 >4%，蓝 ≤4%），每根右侧标精确百分比。

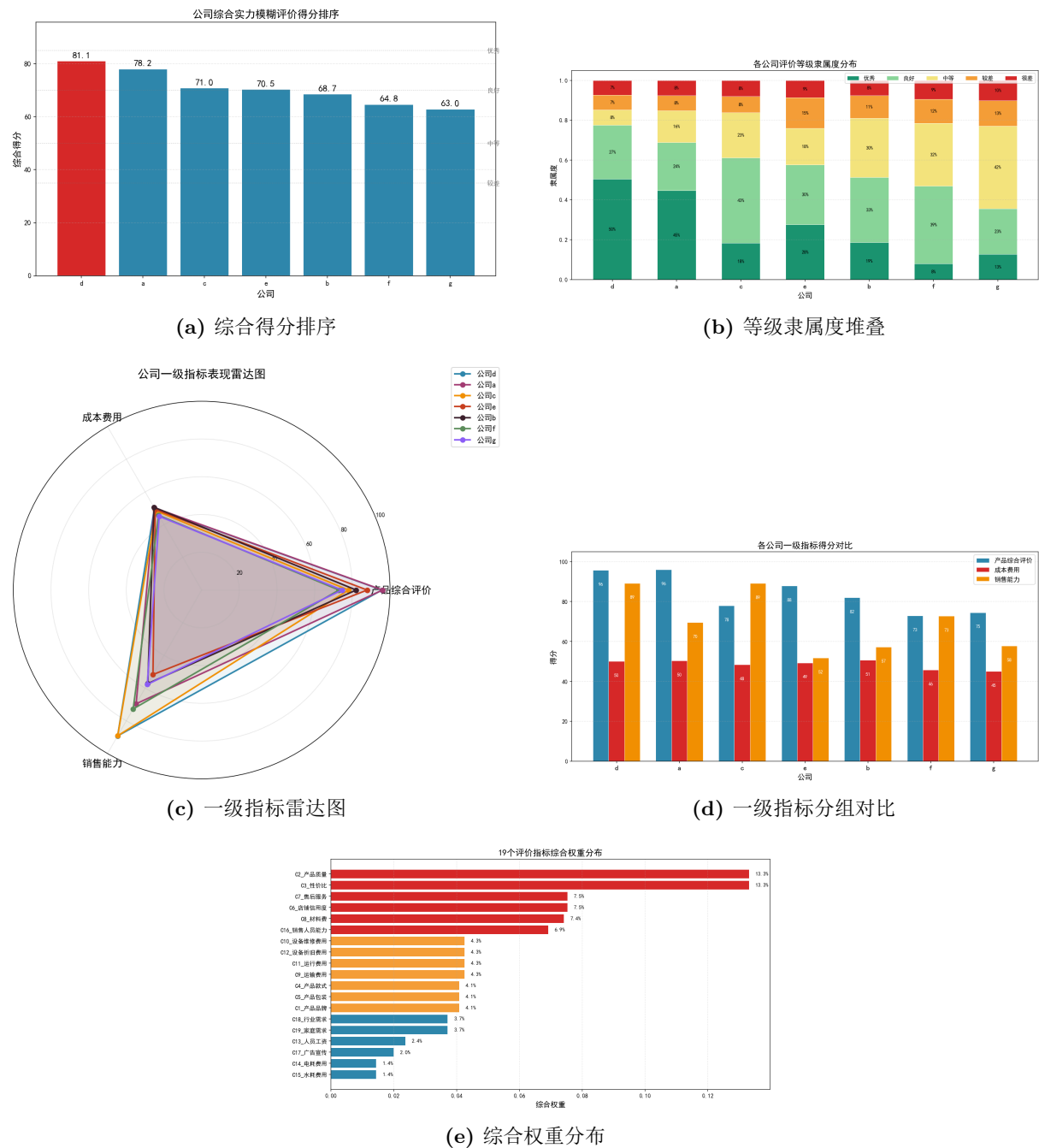


图 4: 五张结果可视化图表

9 代码架构与复用指南

9.1 项目文件结构

项目目录/

company_data.xlsx
fuzzy_evaluation.ipynb
paper.tex / paper.pdf
score_ranking.png
grade_distribution.png
radar_chart.png

原始数据 (3 个工作表)
Jupyter Notebook 主代码
LaTeX 学术论文
图1: 综合得分柱状图
图2: 等级隶属度堆叠图
图3: 雷达图

dimension_comparison.png # 图4：分组柱状图

weight_distribution.png # 图5：权重分布图

9.2 Notebook 模块划分

表 9: Jupyter Notebook 的四模块结构

模块	功能	核心函数	输出
导入库	NumPy/Pandas/Matplotlib + 中文字体	—	—
模块 1	读 Excel → 隶属函数 → 隶属度矩阵	calc_membership	R_matrices
模块 2	判断矩阵 → 方根法 → CR → 权重合成	ahp_weights	w_B, combined_weights
模块 3	两级模糊合成 → 得分排序	—	final_results
模块 4	5 张可视化图表	—	5 个.png 文件

9.3 复用到自己的项目需要改什么？

表 10: 最小改动清单

需要修改	说明
数据文件	替换为你的 Excel，保持相同的工作表结构
GRADE_PARAMS_*	根据你的问题调整各等级 $[a, b, c, d]$ 参数
4 个判断矩阵	根据你的指标重要性重新用 1~9 标度打分
B_groups 字典	根据你的指标分组调整起止索引
GRADE_NAMES, S	如需不同等级名称或分值

其余代码（隶属函数、AHP 方根法、两级合成、可视化）均可直接复用。

10 常见问题与避坑指南

10.1 Q1：隶属函数参数怎么选？

没有标准答案，但有一套可操作的方法：

1. 定”完全隶属区间”的最值（如”90~100 分属优秀”）
2. 定过渡带宽度（通常 10~15 分，太窄则接近刚性分类，太宽则区分度下降）
3. 保证相邻等级有重叠区
4. 在 50、70、85 等分界点附近做抽查，验证隶属度是否合理
5. 如果结果不合理，调整参数后重新跑

10.2 Q2: $CR > 0.1$ 怎么办?

1. 找出判断矩阵中“三角不一致”的地方 ($A > B$, $B > C$, 但 $C > A$)
2. 改掉最明显不合理的那条判断
3. 重新计算 CR
4. 重复直到 $CR < 0.1$

10.3 Q3: 为什么隶属度矩阵里有全 1 或全 0 的行?

检查两点:

- 参数表是否覆盖了整个 0~100 区间?
- 偏小型指标是否误用了 `trapezoid_up`?

10.4 Q4: 能改成四等级或七等级吗?

可以, 但需要注意:

- 同步修改 `GRADE_NAMES`、分值向量 S 、参数表 (3 组 $\times$ n 等级)
- 4 级太粗糙, 7 级以上语义重叠严重
- 建议保持 5 级, 是领域内最成熟的设定

10.5 Q5: 四种模糊合成算子的区别?

算子	类型	使用场景
$M(\cdot, \oplus)$	加权平均型	默认首选, 保留全部信息
$M(\wedge, \vee)$	主因素决定型	数据稀疏, 少数指标突出
$M(\cdot, \vee)$	主因素突出型	同上, 但能利用部分权重
$M(\wedge, \oplus)$	取小有界和	极少使用

10.6 Q6: 如何做灵敏度分析?

- **参数灵敏度**: 将优秀等级的过渡带从 10 分改为 8/12 分, 看排名是否变化
- **AHP 灵敏度**: 将判断标度 ± 1 , 看权重变化幅度
- **权重组合灵敏度**: 固定一级权重, 随机扰动二级权重, 观察排名稳定性

附录：项目文件清单

文件	说明
company_data.xlsx	7 公司 $\times$ 19 指标原始数据
fuzzy_evaluation.ipynb	Jupyter Notebook 完整代码
paper.tex / paper.pdf	LaTeX 学术论文（18 页）
README.md	项目说明
score_ranking.png	综合得分排序柱状图
grade_distribution.png	评价等级隶属度堆叠图
radar_chart.png	一级指标雷达图
dimension_comparison.png	一级指标分组对比柱状图
weight_distribution.png	综合权重水平条形图
模糊综合评价模型建立方法.md	Markdown 版方法指南